

How to calculate X-CA-Signature by using AppSecret

Many customers have some doubts about how Appkey and AppSecret generate signatures when integrating OpenAPI. In this document, we have simplified the entire process into two parts.

First of all, in the first part, I will introduce the process of HCP verification signature when calling OpenAPI commands:

Signature is calculated by AppSecret through Hash calculation. Theoretically, if the appKey and appSecret do not change, the Signature string will not change either. When the client calls an OpenAPI command, it needs to provide the AppKey and the generated Signature in the header of the request:

POST	https://192.168.2.1/artemis/api/common/v1/version					
Params	Authorization	Headers (12)	Body	Pre-request Script	Tests	Settings
☒	Accept					application/json
☒	Content-Type					application/json;charset=UTF-8
☒	X-Ca-Key	← AppKey				24886721
☒	X-Ca-Signature	← Signature				55999898984444
	Key					Value

The following is how to calculate Signature by using an AppSecret to. We will use an appSecret as the key to generate a message digest from the signature string with the HmacSHA256 algorithm, and then base64 encrypt the message digest to generate the Signature:

$$\text{X-Ca-Signature} = \text{Base64}(\text{HmacSHA6}(\text{appSecret}, \text{httpHeaders} + \text{URL}))$$

Generally, httpHeaders+URL can be built like:

“POST

application/json

application/json;charset=UTF-8

/artemis/api/common/v1/version”

Please pay attention to the line breaks in each line. In the last line of URL (the red part), the customer can replace it according to the OpenAPI command actually called.

Test on Postman:

The screenshot displays the Postman interface for a POST request to `https://10.7.84.14/artemis/api/common/v1/version`. The 'Headers' tab is active, showing 12 headers. The 'Pre-request Script' tab is also visible, containing a script that calculates a signature for the request. The 'Body' tab is selected, showing the response in JSON format.

Headers (12)

Key	Value
Host	<calculated when request is sent>
User-Agent	PostmanRuntime/7.26.8
Accept	*/*
Accept-Encoding	gzip, deflate, br
Connection	keep-alive
Accept	application/json
Content-Type	application/json; charset=UTF-8
X-Ca-Key	28034480
X-Ca-Signature	{{SIGNATURE}}
Key	Value

Pre-request Script

```
1 var appSecret = "52bzxtMPf8u7aVohrkXQ";
2 var textToSign = "POST"+"\\n"+"application/json"+"\\n"+"application/json; charset=UTF-8"+"\\n"+" /artemis/api/common/v1/version";
3 console.log(textToSign)
4 var hash = CryptoJS.HmacSHA256(textToSign, appSecret);
5 var signature = hash.toString(CryptoJS.enc.Base64);
6 pm.environment.set("SIGNATURE", signature);
```

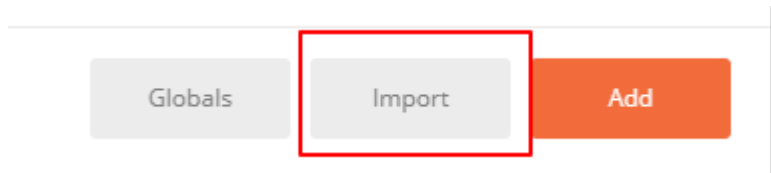
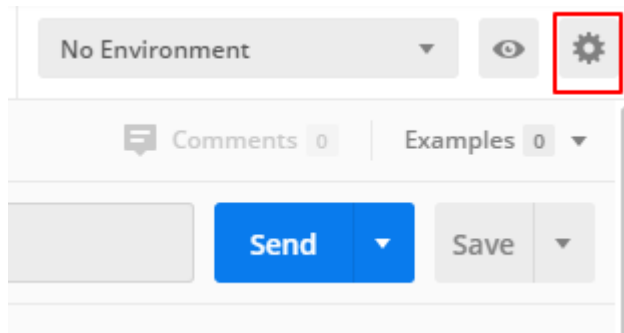
Body

Test Results (1/1)

```
1 {
2   "code": "0",
3   "msg": "Success",
4   "data": {
5     "produceName": "HikCentral Professional",
6     "softVersion": "V2.1.0.0"
7   }
8 }
```

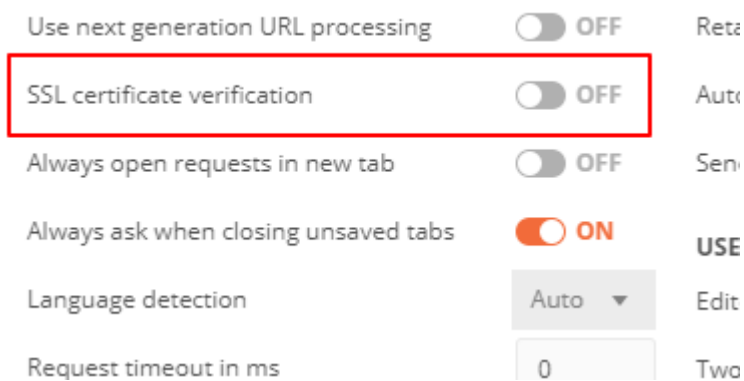
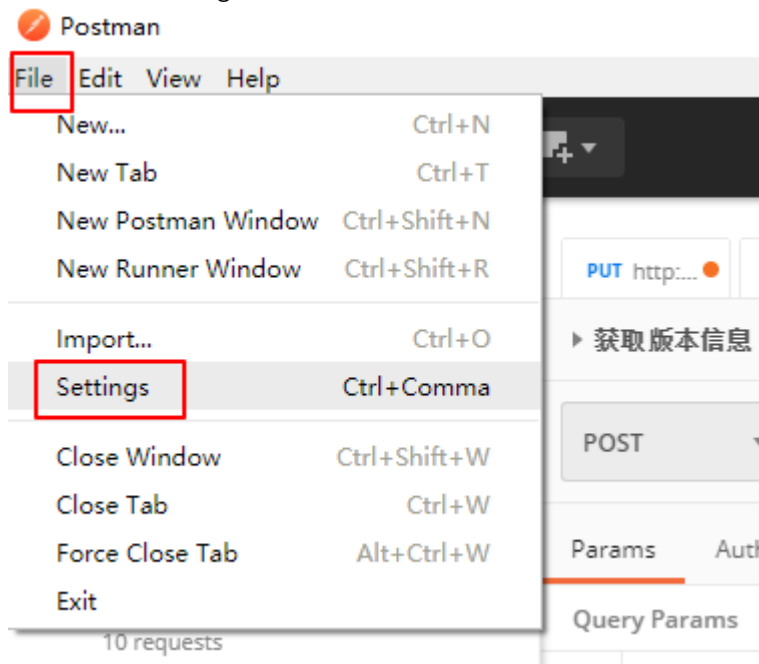
Passed!

Click the “Setting” button on the upper-right, and import the files HCP OpenAPI.postman_collection.json and HCP_OpenAPI.postman_environment.json to the Postman



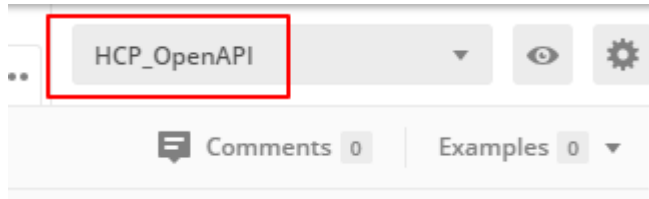
Step 4:

Go to File-Settings, and disable the SSL certificate verification



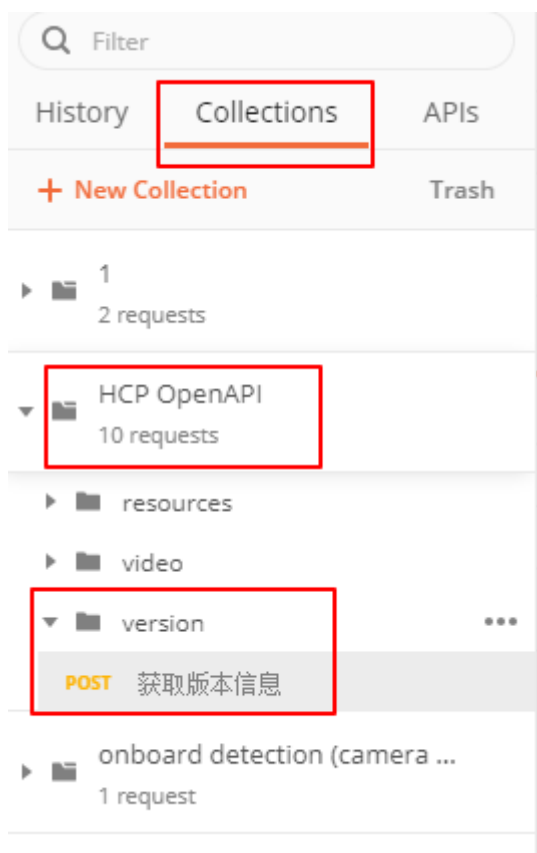
Step 5:

Select the HCP_OpenAPI that on the upper-right as the environment



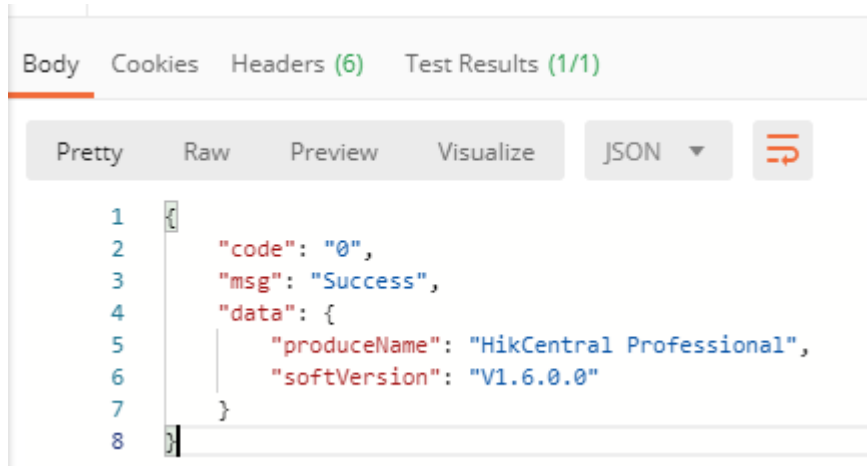
Step 6:

On the left panel, go to Collections-HCP OpenAPI, and here, we select to check the version of OpenAPI as an example:



Step 7:

After click the Send button, if you get the returned data with HCP version successfully, that means your HCP and OpenAPI configurations are completed,



Step 8:

If you want to know how to use the appKey and appSecret to create the X-CA-Signature, please check the following two parts on Postman:

a. Headers

Params	Authorization	Headers (12)	Body	Pre-request Script	Tests	Settings
Headers 8 hidden						
		KEY	VALUE			
☒		Accept	application/json			
☒		Content-Type	application/json;charset=UTF-8			
☒		X-Ca-Key	{{AK}}			
☒		X-Ca-Signature	{{SIGNATURE}}			
		...	...			

b. Pre-request Script

```
1 var appSecret = pm.variables.get("SK");
2 var textToSign = "";
3 textToSign += request.method + "\n";
4 textToSign += request.headers.accept + "\n";
5 textToSign += request.headers["content-type"] + "\n";
6 var url_path = "/artemis" + request.url.split("artemis")[1];
7 url_path = url_path.replace("{{API_VER}}", pm.variables.get("API_VER"));
8 textToSign += url_path;
9 var hash = CryptoJS.HmacSHA256(textToSign, appSecret);
10 var signature = hash.toString(CryptoJS.enc.Base64);
11 pm.environment.set("SIGNATURE", signature);
```

Step 9: Check the Postman Console, and find out the value of X-CA-Signature

